

```

1 // Problème 1
2
3 public class LoadBalancer extends Thread {
4     static String hosts[] = {"host1", "host2"};
5     static int ports[] = {8081, 8082};
6     static int nbHosts = 2;
7     static Random rand = new Random();
8     Socket cli;
9
10    public LoadBalancer (Socket s) {
11        cli = s;
12    }
13
14    public static void main (String args[]) {
15        ServerSocket ss = new ServerSocket(8080);
16        while (true) {
17            Socket s = ss.accept();
18            // Important de ne pas faire .run() !!!
19            new LoadBalancer(s).start();
20        }
21    }
22
23    public void run() {
24        try {
25            int nServ = rand.nextInt(nbHosts);
26            Socket serv = new Socket(hosts[nServ], ports[nServ]);
27            InputStream cliIS = cli.getInputStream();
28            InputStream serv_is = serv.getInputStream();
29            OutputStream serv_os = serv.getOutputStream();
30            byte buff[] = new byte[1024];
31
32            int nb = cli_is.read(buff,0,1024);
33            serv_os.write(buff,0,nb);
34            nb = serv_is.read(buff,0,1024);
35            cli_os.write(buff,0,nb);
36            cli.close();
37            serv.close();
38        } catch (IOException e) {
39            e.printStackTrace();
40        }
41    }
42 }
43
44
45
46 // Problème 2
47 public interface Callback extends Remote {
48     public void onMessage(Message m) throws RemoteException;
49 }
50
51 public class CallbackImpl extends UnicastRemoteObject implements Callback {
52     public CallbackImpl() throws RemoteException {}
53 }

```

```

54     public void onMessage(Message m) throws RemoteException {
55         System.out.println(m);
56     }
57 }
58
59 public interface MOM extends Remote {
60     public void publish(String topic, Message m) throws RemoteException;
61     public void subscribe(String topic, Callback callback) throws RemoteException;
62 }
63
64 public class MOMImpl extends UnicastRemoteObject implements MOM {
65     Hashtable<String, ArrayList<Callback>> table;
66
67     public MOMImpl() throws RemoteException {
68         table = new Hashtable<String, ArrayList<Callback>>();
69     }
70
71     public void subscribe(String topic, Callback callback) throws RemoteException {
72         ArrayList<Callback> l = table.get(topic);
73         if (l == null) {
74             l = new ArrayList<Callback>();
75             table.put(topic, l);
76         }
77         l.add(callback);
78     }
79
80     public void publish(String topic, Message m) throws RemoteException {
81         ArrayList<Callback> l = table.get(topic);
82         for (Callback callback : l) {
83             callback.onMessage(m);
84         }
85     }
86
87     public static void main (String args[]) {
88         LocalRegistry.createRegistry(8081);
89         Naming.bind("//localhost:8081/mom", new MOMImpl());
90     }
91 }
92
93 public class Client {
94     public static void main (String args[]) {
95         // Important de caster en MOM et pas MOMImpl
96         MOM m = (MOM)Naming.lookup("//localhost:8081/mom");
97         m.subscribe("sujet1", new CallbackImpl());
98         m.publish("sujet1", "Un petit message :)");
99     }
100 }
101

```